

ENGINEERING HIRING / TECHNICAL ASSESSMENT

Backend Engineering Assessment

Retail data integration, exception detection & task reliability

TIMEBOX	STACK	FORMAT
4-6 hours	Python / FastAPI PostgreSQL	Take-home + technical walkthrough

Candidate brief

Built around a realistic OPSiB workflow. Synthetic datasets are supplied separately.

The assessment rewards sound decisions, failure handling, tests and clear reasoning - not code volume.

THE CHALLENGE / 01

Build a reliable slice of OPSiB

OPSiB connects retail systems, detects operational exceptions and makes sure the right person investigates and resolves them. The pilot is not a dashboard exercise. It must turn inconsistent source records into a traceable finding and a task that a real employee can act on.

Your scenario

A retailer has exported purchase-order lines from its ERP, goods-receipt lines from its warehouse system, product and location mappings, and employee/shift data from HRMS and workforce management. The systems use different product and location identifiers. Some records are duplicated; some receipts are partial; one order is cancelled; one expected-receipt window has not closed; one PO uses cartons while the receipt uses individual units.

> Your job: Build a small backend service that imports and validates these datasets, reconciles them into a consistent model, detects valid receipt shortfalls, routes tasks using employee eligibility, and records task progress with an auditable history.

What we are looking for

Data judgment: You distinguish field mapping from entity identity and business meaning. You do not force an ambiguous field into a guessed interpretation.

Correctness: Repeated imports do not double-count stock or create duplicate tasks. Late and partial data are handled intentionally.

Operational thinking: A receipt shortfall is not automatically proof of supplier fault or financial loss.

Engineering quality: Clear module boundaries, persistence, useful tests and explicit failure behaviour.

Ownership: You can explain the trade-offs and make a small change to your own implementation during review.

Timebox and stack

Target effort is 4-6 hours. Use Python with FastAPI and PostgreSQL, or explain a close alternative. A full frontend is not required. You may use an ORM, migration tool and background worker if useful, but avoid adding infrastructure that does not improve this bounded task.

AI coding tools are allowed. You are responsible for every submitted line and must disclose the tools you used. The follow-up review will test understanding and the ability to modify the implementation.

INPUT DATA / 02

Data pack and source assumptions

Download the companion fixture ZIP. All data is synthetic. Use the frozen evaluation time in the fixture README: 2026-10-16 15:00:00 Asia/Kolkata. Do not use the machine clock for scenario logic.

File	What it represents	Important detail
purchase_order_lines.csv	ERP PO lines	Contains an identical duplicate row. PO status and expected receipt deadline matter.
goods_receipt_lines.csv	WMS receipt lines	Contains an identical duplicate source record. Receipts may be split across GRNs.
product_aliases.csv	Product identity and UOM map	ERP and WMS IDs can map to one canonical SKU. BOX converts to 12 EA for one product.
location_aliases.csv	Location crosswalk	ERP, WMS and HRMS codes may identify the same OPSiB store.
employees.csv	HRMS roster	Employment status, role, location and manager relationship.
workforce_status.csv	Shift/availability snapshot	An inactive employee appears ON_SHIFT; employment eligibility must still win. One active receiver is OFF_SHIFT. Regional roles may be region-scoped without a store location.

A deliberate limit in the evidence

The fixture pack contains no supplier dispatch / ASN data. It can support a receipt shortfall against the PO, but it cannot prove that the supplier dispatched fewer units, that a carrier lost them, or where the shortage occurred. Keep the finding neutral and identify the evidence still needed to establish cause.

Canonical concepts you may use

Concept	Suggested canonical fields
Product	canonical_product_id, display_name (optional), base_uom, status
Location	canonical_location_id, region_id, status
PO line	source identity, PO + line, product, location, supplier, ordered quantity in base UOM, status, deadline
Receipt line	source identity, receipt + line, PO + line, product, location, accepted quantity in base UOM, status, received time
Employee / availability	employee ID, active status, role, canonical location, manager ID, shift status, observed-at time
Exception / task	tenant, stable case key, rule version, evidence refs, status, assignee, due time and audit events

These are concepts, not a mandatory schema. Use names and structures you can defend, and document any simplification. Retain enough source identity and lineage to explain where every finding came from. Employee location may be optional for region-scoped roles; model organizational scope deliberately.

REQUIRED WORK / 03

Task A - Ingestion, mapping and persistence

Implement a repeatable way to load all supplied CSVs and turn them into validated records. This may be a CLI plus API, or an API-first implementation. Do not hard-code the final answer for each PO.

Required behaviour

1. Register and validate imports. Report accepted, rejected and duplicate rows. Invalid rows must be visible with a useful reason; do not silently drop them.
2. Normalize identifiers. Map source product and location IDs through the alias files. Employee locations must be mapped before they can be used for routing.
3. Normalize units. Convert quantities to a declared base unit before comparison. Do not compare 10 BOX directly with 96 EA.
4. Preserve provenance. Retain source system, source record ID, input/import reference and the mapping/rule version used. A reader must be able to trace a finding to source rows.
5. Make ingestion idempotent. Importing the same files twice must not double-count receipt quantities or duplicate canonical source facts. If the same source ID arrives later with conflicting values, flag it for review rather than silently overwriting it.
6. Keep business state separate from raw input. Explain which system is authoritative for each field and how you handle a missing or ambiguous mapping.

Engineering expectations

Use typed validation and explicit timezone parsing. Use the frozen evaluation time rather than the current clock.

Use database constraints/transactions where they protect business invariants. If you choose in-memory or SQLite shortcuts, state the limitations and what would change for a live pilot.

Provide a usable import result, not only console output. A response should make partial success and row-level errors clear.

Keep source-specific parsing/mapping separate from exception rules so another retailer export can be added without rewriting the detector.

Suggested API surface

`POST /imports/{dataset}` - validate and load a dataset
`GET /imports/{import_id}` - show counts, errors and processing status
`GET /data-quality` - show unresolved mappings / rejected records

> Design question: How do you prevent a second import or concurrent worker from creating a different result from the first import? Explain the database constraint or transaction boundary that makes your answer true.

REQUIRED WORK / 04

Task B - Reconciliation and exception detection

Implement a deterministic receipt-shortfall rule. The rule must use mapped product/location identity, source status, receipt posting status, comparable units, receipt timing and duplicate-safe aggregation.

Rule definition to implement

For an eligible, non-cancelled PO line, calculate expected and accepted quantities in the product base unit. Consider the line actionable only after its expected-receipt deadline has passed at the frozen evaluation time. Compare accepted, posted receipt lines linked to that PO line. Use a zero tolerance unless you explicitly implement a configurable tolerance. A line with incomplete or ambiguous mapping is a data-quality issue, not a business exception.

Required scenario outcomes

Scenario	Required interpretation
PO-501	100 EA ordered; split receipts of 60 + 32 EA; deadline passed. One shortfall of 8 EA.
PO-502	40 EA ordered and 40 EA accepted before/at the expected close. No shortfall exception.
PO-503	50 EA ordered; 20 EA received but deadline is still in the future. Do not raise an actionable shortfall at the frozen evaluation time.
PO-504	PO line is CANCELLED. Do not raise a shortfall for its missing receipt.
PO-505	10 BOX ordered; conversion is 12 EA per BOX; 96 EA accepted. Compare 120 EA expected with 96 EA received: shortfall is 24 EA, once deadline passes.
PO-506	20 EA ordered; one 10 EA receipt record is repeated with the same source ID. Count it once: shortfall is 10 EA, not 0 EA.

What a finding must contain

A stable exception ID and idempotent business key, plus type, status and detection timestamp.

The PO/line, canonical product and location, expected quantity, received quantity, unit and computed variance.

Linked source record IDs and the rule/mapping versions used to compute the result.

A neutral explanation of what is known and what remains unknown. The absent dispatch data means supplier fault must not be asserted.

An explicit path for cases that cannot be evaluated because evidence is stale, missing, conflicting or unmapped.

Do not solve this by comparing row counts or matching on product name alone. Receipts can be partial, source rows can repeat, IDs differ across systems, and a PO line can have several receipt lines. Show how your implementation preserves the correct grain of each record.

REQUIRED WORK / 05

Task C - Employee routing and task lifecycle

An exception is a business case; a task is work that a person performs. Keep them separate. One exception may require multiple tasks, and a task can be reassigned without creating a second exception.

Assignment rules

1. For shortfall cases at STORE-014, the primary eligible role is receiving_associate at the same canonical location.
2. An employee must be employment-status ACTIVE. If workforce status is required by your policy, the employee must also be ON_SHIFT at the frozen evaluation time, and the availability sample must be fresh enough for your chosen rule.
3. HR-101 is active but OFF_SHIFT. HR-103 is marked INACTIVE even though the shift export says ON_SHIFT. Neither should be assigned.
4. HR-102 is the eligible on-shift receiving associate at STORE-014 and should be selected for a task there.
5. For a task at STORE-015, no eligible receiving associate is supplied. Fall back to the active, on-shift store manager HR-401. If no approved fallback exists, keep the task visible as needing manual assignment; do not silently assign an ineligible person.
6. Persist why a person/queue was selected, and preserve assignment history. If you implement a different valid policy, document it and explain the safety invariants.

Task lifecycle

Implement an explicit, server-enforced state transition model. A reasonable minimum is `CREATED -> ASSIGNED -> ACKNOWLEDGED -> IN_PROGRESS -> COMPLETED -> PENDING_VERIFICATION -> VERIFIED`, with documented blocked/rejected/reopened behaviour where needed. Do not accept arbitrary status changes from the client.

Required operations

- List exceptions and inspect a single exception with its supporting source records.
- List tasks for an assignee or location, with status and due time.
- Acknowledge/start/complete a task through validated state transitions.
- Reassign a task with a reason and record who made the change.
- Capture a resolution code and evidence reference; only mark an exception resolved when the defined verification criteria are satisfied.
- Keep an audit/event history for every important state transition and assignment change.

Suggested endpoints (not a route-name requirement)

`GET /exceptions` · `GET /exceptions/{id}` · `GET /tasks?assignee_id=...`

`POST /tasks/{id}/acknowledge` · `POST /tasks/{id}/complete` · `POST /tasks/{id}/reassign`

> Design question: If the API commits task completion but the client times out before receiving the response, what happens when the client retries? Describe and implement a safe answer.

REQUIRED WORK / 06

Task D - Reliability, tests and AI boundary

This part is where we learn whether the implementation will survive messy data and real operational use rather than only passing the happy path.

Reliability requirements

Use database transactions and appropriate uniqueness constraints for imported source identities and task creation. Explain how concurrent/retried processing avoids duplicate business effects.

Handle conflicting duplicate source IDs explicitly. An identical repeat may be idempotent; the same ID with changed business content needs a documented update/version/conflict policy.

Keep timestamps timezone-aware and use the supplied frozen time for the deadline test.

Define useful API errors and structured logs. Avoid logging credentials or unnecessary personal data.

Explain what changes for production: tenant-level authorization, secure secret storage, migrations, monitoring, retention and integration credentials. Do not claim these are implemented if they are only documented.

Minimum tests

1. Unit tests for UOM conversion, partial receipts, deadline boundaries and cancelled PO lines.
2. Integration test that loads the fixture pack and verifies the required scenario outcomes.
3. Idempotency test: import the exact same receipt and PO data twice and assert the business result does not change.
4. Identity/assignment tests covering source-location mapping, off-shift employee, inactive employee and fallback manager.
5. Workflow tests for invalid transitions, repeated completion requests and audit history.
6. At least one failure-path test, such as unknown product mapping, conflicting duplicate ID or absent eligible assignee.

Claude API: optional but design for it

You do not need a live Claude API key and should not spend money on API calls for this exercise. Describe or implement a small provider interface that could later accept a structured evidence package and return a validated investigation summary.

Claude may explain evidence, identify missing context and suggest investigation steps. It must not calculate authoritative quantities, invent source records, decide employee permissions, perform unapproved ERP writes or mark a business outcome as verified. The application must validate structured output and remain functional if the model times out or returns invalid data.

> One good design: a deterministic detector produces a typed finding and evidence IDs; an optional AI service receives only the necessary case context; structured output is schema-checked and every cited record ID is checked against the supplied evidence before display.

SUBMISSION / 07

What to submit

Keep the implementation deliberately small. We prefer a well-tested vertical slice and honest boundaries over a broad but fragile architecture.

Required deliverables

Source code with a clear module layout and sensible names.

README with setup/run instructions, environment variables, migrations and the commands needed to run tests.

Data model / schema and short architecture note explaining ingestion, canonical mapping, detection, routing and persistence.

API contract or OpenAPI link/export, with example requests and responses.

Automated tests and the command/output summary used to run them.

Assumptions and trade-offs, including what is intentionally omitted and what would be hardened for a live retailer.

AI tooling disclosure: tools used and which parts you independently reviewed or changed.

Implementation constraints

Do	Avoid
Use the supplied files as inputs and preserve them unchanged.	Hard-code exception IDs, expected totals or assigned people as the result.
Use configuration and mapping data where appropriate.	Join by display name or SKU text similarity without explaining identity rules.
Make failure states visible and recoverable.	Silently drop bad rows, swallow errors or return success for partial failure.
Keep the detector deterministic and independently testable.	Use Claude to do exact arithmetic or decide who is authorized.
Build a small maintainable service with a real persistence boundary.	Spend most of the time on a frontend, microservices, Kafka deployment or an elaborate UI.

Suggested repository layout

`app/api/ · app/ingestion/ · app/mapping/ · app/detection/ · app/routing/ · app/workflows/ · app/db/ · tests/ · migrations/`

This is only a suggestion. A different layout is fine if responsibilities remain easy to understand and test.

REVIEW PROCESS / 08

How the assessment will be reviewed

After submission, expect a 30-45 minute technical walkthrough. You will be asked to explain the design in your own words and make a small change to the code. The discussion is part of the assessment, not a trick: it helps us understand your decisions and how you work with an unfamiliar production problem.

Be ready to discuss

- Why you chose your canonical data model and how it handles source IDs that change or collide.
- What happens if a receipt arrives late, a record is repeated, an import partially fails or a mapping is ambiguous.
- How your code avoids two workers creating two tasks for the same business issue.
- Why the result is a receipt shortfall rather than a proven supplier fault.
- How you would add another retailer export without changing the rule engine.
- What needs to change before accepting production data from multiple tenants.

Evaluation areas

Area	Weight	Evidence we look for
Correctness and business reasoning	25%	Expected scenarios, UOM, partial receipt, deadline, cancellation and neutral claims about cause.
Data mapping and provenance	20%	Crosswalks, canonical model, traceability, conflicting IDs and row-level validation.
Reliability and idempotency	20%	Duplicate/retry safety, transaction boundaries, conflicts and failure recovery.
Workflow and employee routing	15%	Eligibility, fallback, valid state transitions, audit and resolution verification.
Code quality and test design	15%	Separation of concerns, readable implementation, meaningful unit/integration tests.
Communication and ownership	5%	Clear README, explicit assumptions and ability to explain and change the code.

Submission checklist

- Can another engineer run the service and tests from a clean checkout?
- Are the six required scenario interpretations tested?
- Can every actionable finding be traced to source records and a rule version?
- Can the candidate explain the failure modes and trade-offs without relying on an AI tool?

OPSiB | Operational intelligence that turns exceptions into accountable action.

Please do not include real retailer credentials, customer data, employee PII or API secrets in the submission. Use only the supplied synthetic fixtures.